

TASK APPLICATION:OWASP,WebGoat v8.x

TESTING TOOL:OWASP ZAP(zed attack Proxy)

INTRODUCTION

This assessment documents the identification, validation, and manual exploitation of three critical application security vulnerabilities within the WebGoat training environment. Testing was executed using an automated vulnerability scanner

(OWASP ZAP) followed by manual proof-of-concept validation to confirm the impact of each vulnerability. All vulnerabilities analyzed represent severe risks listed in the OWASP Top 10 vulnerabilities documentation.

VULNERABILITIES

WebGoat was accessed on Localhost:8080

It should be noted that webgoat was started on my PC Terminal before i was able to access it on the browser.

```
Cd downloads
```

```
Java -jar webgoat-2025.3.jar
```

(webgoat file was downloaded on the PC as well)

On the browser:

```
http://localhost:8080/WebGoat
```

1.SQL INJECTION.

Using webgoat to check for vulnerabilities,the SQLi (intro) section was used. The first four pages were analysed to have an in-depth knowledge of SQLi and vulnerabilities attached.

On page 5 of the SQLi section on WebGoat,the following query was injected.

```
GRANT ALL PRIVILEGES ON grant_rights TO unauthorized_user;
```

This command allocates broad administrative capabilities(such as read,write modify) So that the SQLi can be imputed by the attacker.

Next step was to inject the SQLi through the selections used in the image below to gain access to users from the lists of the company used in the example on webgoat.

Releases · WebGoat/WebGoat x WebGoat

localhost:8080/WebGoat/start.mvc?username=jessica10#lesson/SqlInjection.les...

SQL Injection (advanced)
SQL Injection (mitigation)
Cross Site Scripting
Cross Site Scripting (stored)
Cross Site Scripting (mitigation)
Path traversal

(A5) Security Misconfiguration >
(A6) Vuln & Outdated Components >
(A7) Identity & Auth Failure >
(A8) Software & Data Integrity >
(A9) Security Logging Failures >
(A10) Server-side Request Forgery >
Client side >
Challenges >

Try It! String SQL injection

The query in the code builds a dynamic query as seen in the previous example. The query is built by concatenating strings making it susceptible to String SQL injection:

```
"SELECT * FROM user_data WHERE first_name = 'John' AND last_name
```

Try using the form below to retrieve all the users from the users table. You should not need to know any specific user name to get the complete list.

```
SELECT *  
FROM  
user_data  
WHERE  
first_name  or  '  
= 'John'  
AND  
last_name  
= '
```

Get Account Info

This query opened the list of the employees in the company and their information.

Releases · WebGoat/WebGoat x WebGoat

localhost:8080/WebGoat/start.mvc?username=jessica10#lesson/SqlInjection.les...

AND
last_name
= '

You have succeeded:
USERID, FIRST_NAME, LAST_NAME, CC_NUMBER, CC_TYPE, COOKIE, LOGIN_COUNT,
101, Joe, Snow, 987654321, VISA, , 0,
101, Joe, Snow, 2234200065411, MC, , 0,
102, John, Smith, 2435600002222, MC, , 0,
102, John, Smith, 4352209902222, AMEX, , 0,
103, Jane, Plane, 123456789, MC, , 0,
103, Jane, Plane, 333498703333, AMEX, , 0,
10312, Jolly, Hershey, 176896789, MC, , 0,
10312, Jolly, Hershey, 333300003333, AMEX, , 0,
10323, Grumpy, youaretheweakestlink, 673834489, MC, , 0,
10323, Grumpy, youaretheweakestlink, 33413003333, AMEX, , 0,
15603, Peter, Sand, 123609789, MC, , 0,
15603, Peter, Sand, 338893453333, AMEX, , 0,
15613, Joesph, Something, 33843453533, AMEX, , 0,
15837, Chaos, Monkey, 32849386533, CM, , 0,
19204, Mr, Goat, 33812953533, VISA, , 0,

IMPACT/DANGER:High.The database bypassed standard authentication filtering logic,causing mass data exposure by leaking the entire internal user account table.

2.CROSS-SITE SCRIPTING(XSS).

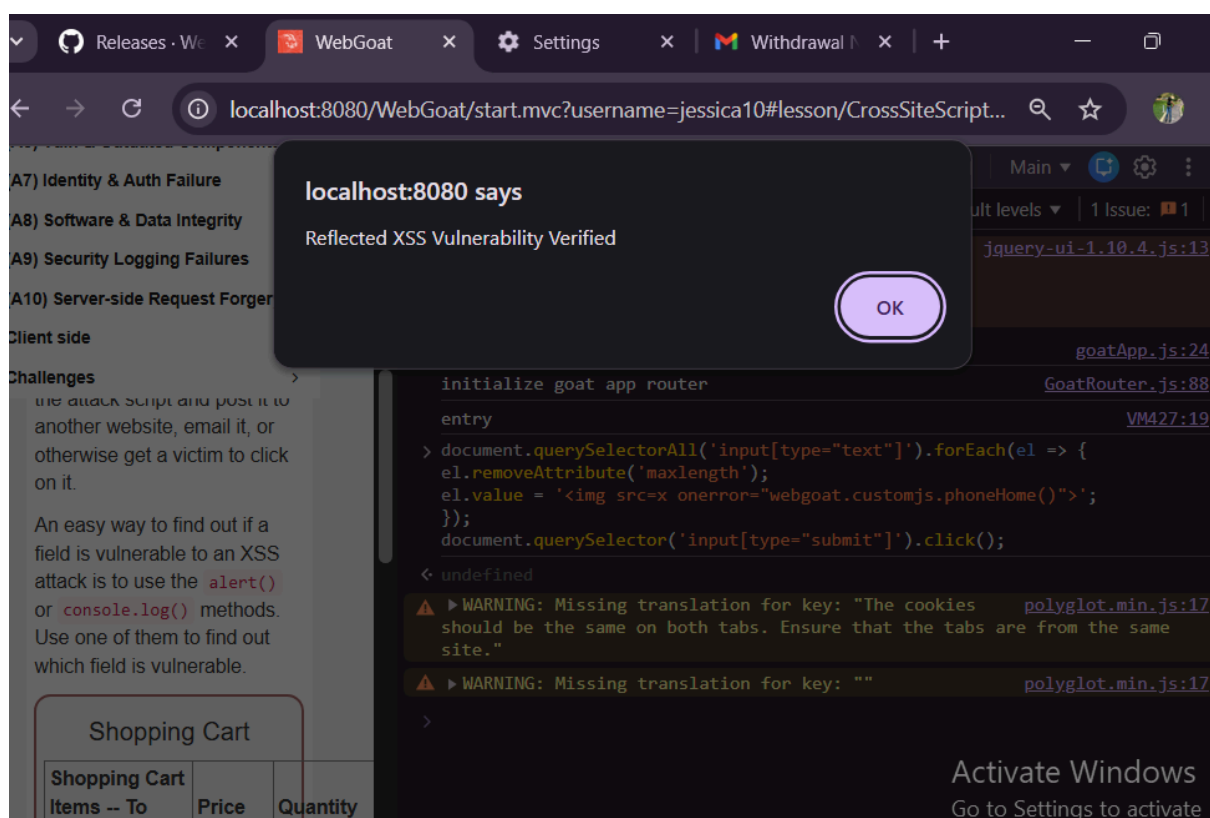
This was identified through manual input validation testing on text input fields within the application's shopping cart submission module.

Exploitation process:Navigated to the Reflected XSS shopping cart module on WebGoat .The application implemented a weak client side length filter (maxlength)and regular expression script filter.This constraint was stripped by modifying input configurations,and a malicious HTML event-driven payload was submitted,in the query section.

```

```

It returned the following prompt on the browser



IMPACT/DANGER:Medium-High,Attackers can hijack sessions,steal authentication state cookies or force arbitrary script execution in a browser context.

3. Cross-Site Request Forgery (CSRF)

This step involved the stimulation of an external attacker forcing the logged-in browser to post fake comments on WebGoat.

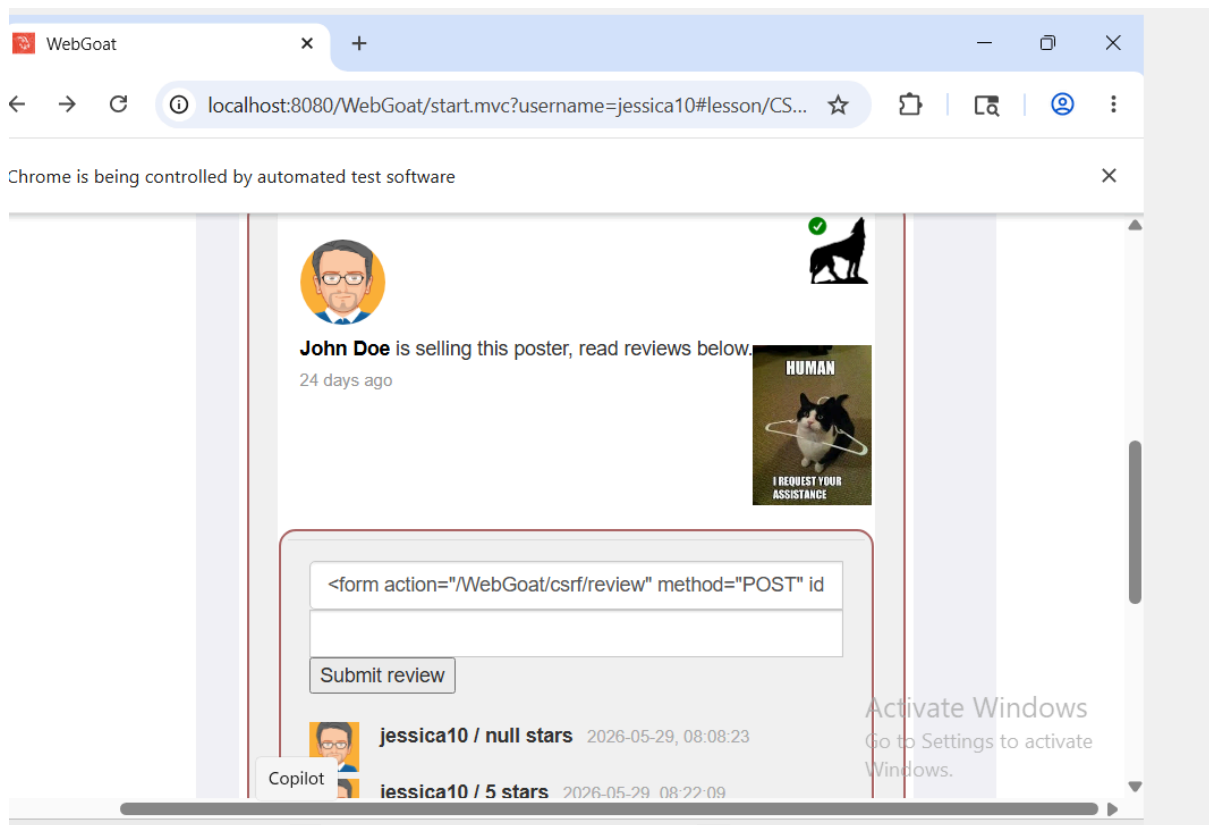
The application accepted state-modifying requests without validating origin credibility or tying state changes to unguessable tokens.

Navigated to the CSRF component. A structured, automated HTML transaction payload form was crafted to trigger background processing:html

```
<form action="http://localhost:8080/WebGoat/csrf/review"
method="POST" id="csrfForm">
    <input type="hidden" name="reviewText" value="This is a
fake comment forced by an external attacker!" />
    <input type="hidden" name="stars" value="5" />
</form>
<script>
    document.getElementById('csrfForm').submit();
</script>
```

In between OWASP ZAP intercepted and analysed also automate the exploitation or testing of this CSRF vulnerability.

After using the HTML code some forged comment was posted on the WebGoat page.



IMPACT/DANGERS:

Real-World Dangers
Reputation Manipulation: Forcing thousands of real users to automatically post fake 5-star reviews or malicious comments (exactly like your WebGoat task)

Unauthorized Purchases: Forcing a logged-in user to silently click an invisible "Buy Now" or "Transfer Funds" button on an e-commerce or banking site.

STEPS FOR VULNERABILITY MITIGATIONS

SQLi:

1. Implementation of Parameterized queries, to completely isolate user inputs from executable SQL code.
2. Apply principle of least privilege to restrict database user permissions to only the essential tables and actions.

CROSS-SITE SCRIPTING(XSS):

- 1.Implement **Context -Aware Output Encoding** to safely sanitize all user-supplied data before rendering it in the browser
- 2.Restrict the sources from which scripts can be executed
- 3.Use modern frameworks e.g REACT

Cross-Site Request Forgery (CSRF):

- 1.Generate unique, cryptographically strong tokens for every state-changing request.
 - 2.SameSite Cookie Attribute: Set cookies to SameSite=Strict or SameSite=Lax to control cross-site sharing.
 - 3.Re-Authentication: Require passwords or multi-factor authentication for high-risk actions.
- Custom Request Headers: Use frameworks that attach custom headers to verify the request origin.